

Informe Técnico del Sistema de Gestión de Pacientes e Historias Clínicas

Grupo62_Paciente--HistoriaClinica

El trabajo práctico integra una solución de consola para administrar pacientes y sus historias clínicas mediante operaciones CRUD completas con *eliminación lógica (soft delete)*, capa de servicios y persistencia *JDBC hacia MySQL*. La consigna enfatiza la *relación uno a uno (1:1)* entre paciente e historia clínica, la *arquitectura en cuatro capas* y el uso de *transacciones coordinadas* para garantizar integridad y trazabilidad.

El código actual demuestra principios de diseño profesional: los modelos **Paciente** e **HistoriaClinica** heredan de una clase base con soporte de *soft delete*, los servicios encapsulan reglas de negocio, los DAOs centralizan SQL parametrizado y las operaciones de alto nivel se orquestan desde un menú interactivo que puede ejecutarse tanto desde IDE como desde línea de comandos.2. Integrantes y roles

Integrante	Tareas
SANTIAGO CAICIA MASSELLO	DAOs, JDBC, Prepared Statement.
GUILLERMO CAMPOY	Setup inicial, documentación, menú, UI
LUIS CISNEROS	Diagrama UML Modelo Paciente-HC
NICOLAS ALBERTO COLMAN	Services (Generic, Paciente, Historia Clínica)

Elección del dominio y justificación

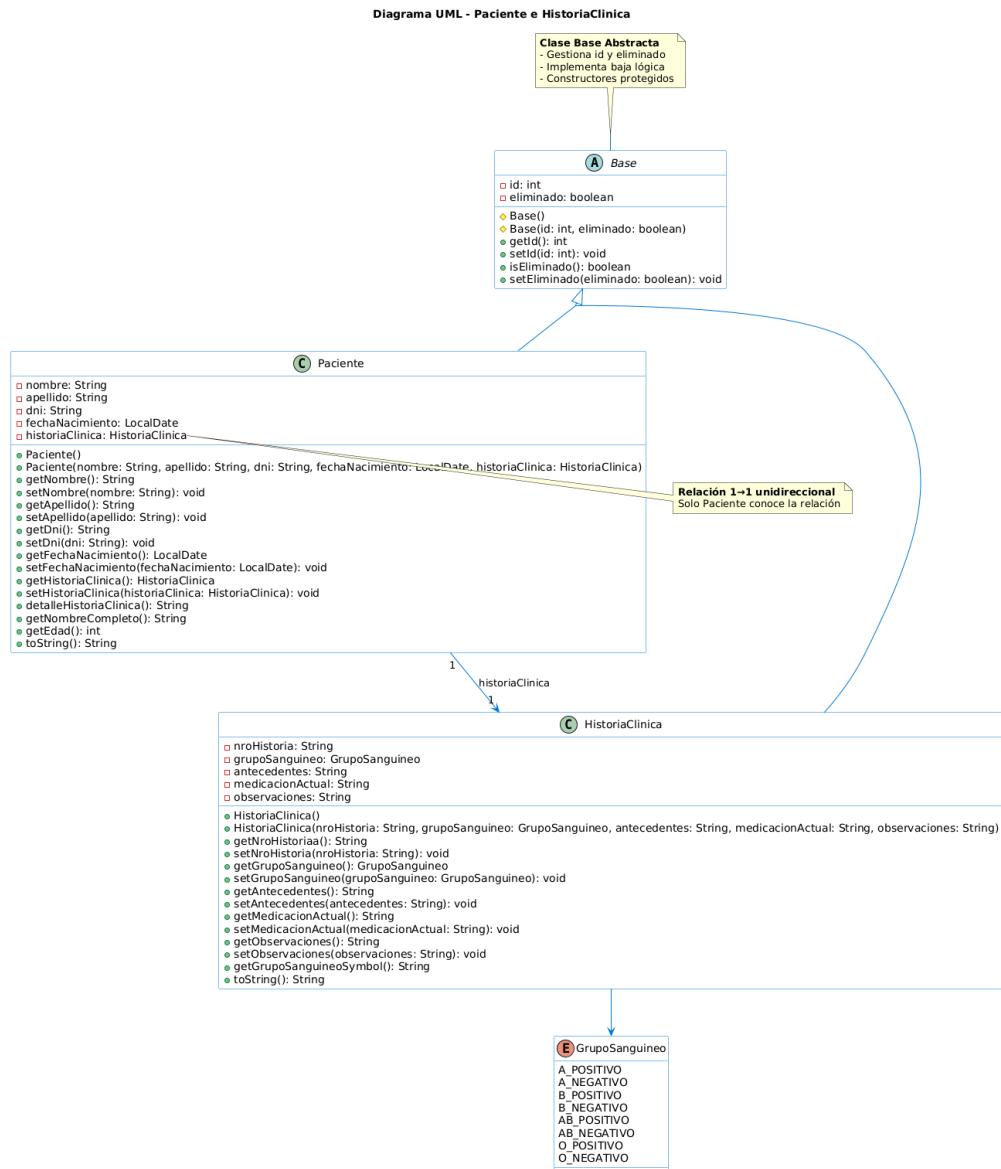
- **Descripción breve del dominio:** *El equipo seleccionó de mutuo acuerdo el caso de historia clínica - paciente para su desarrollo, considerándolo particularmente desafiante y relevante para los futuros retos profesionales, a pesar de que todos los casos compartían los mismos criterios. El grupo lo percibió como una excelente oportunidad para aplicar de manera más rigurosa y ambiciosa los conocimientos adquiridos en la asignatura. Cabe destacar que este mismo equipo fue el encargado de desarrollar el trabajo integrador de base de datos.*
- **Dolores/responsabilidades resueltas:** *Si bien esto no es estrictamente propio del trabajo en sí mismo, es relevante mencionar el tiempo dedicado y el trabajo asíncrono, consecuencia de otras entregas de otras materias, al ser un equipo consolidado a lo largo del año, tenemos comunicación fluida, canales específicos por materia y en lo que es estrictamente relacionado al desarrollo, tenemos una forma esquematizada para trabajar en los repositorios, con modelos claros de responsabilidades, ramas y merges frecuentes a rama main, lo que permite que el resto del equipo vaya adecuando sus cambios con los de sus compañeros.*
- **Impacto esperado:** *El gran impacto particularmente en esta entrega, es el nivel de desafío que conlleva la misma, un trabajo integrador que demuestra lo que aprendimos durante la materia, como lo adaptamos y llevamos a la práctica, tomando como base los proyectos modelo entregados por los docentes, entendiendo los mismos y adecuándolos a nuestras necesidades, manteniendo las estructuras y reglas de diseño y arquitectura planteadas como parte de la entrega*

Diseño del dominio4.1 Decisiones clave (1→1, FK única vs PK compartida)

- **Relación paciente–historia:** el modelo opta por una relación **1→1** donde la historia clínica mantiene una restricción **UNIQUE** sobre **paciente_id**, evitando que un paciente tenga más de una historia activa y simplificando consultas agregadas.
- **Clave principal independiente:** tanto **paciente** como **historias_clinicas** poseen PK autoincrementales, lo que facilita auditorías y operaciones independientes aun cuando se realicen bajas lógicas coordinadas; la unicidad se refuerza mediante llaves naturales (**dni**, **nro_historia**).
- **Soft delete transversal:** la clase abstracta **Base** provee el campo **eliminado** heredado, permitiendo gestionar auditorías y listados específicos sin perder registros sensibles para el dominio clínico.
- **Valor agregado en modelos:** **Paciente** incorpora el helper **detalleHistoriaClinica()** para emitir reportes legibles, mientras **HistoriaClinica** define una enumeración **GrupoSanguineo** que estandariza el catálogo y simplifica validaciones futuras.

UML de referencia

(se encuentra en la raíz del repositorio github)



*El siguiente diagrama resume las entidades principales, sus atributos y asociaciones.
Sirve como contrato de diseño para las capas superiores.*

Arquitectura por capas

La implementación respeta una arquitectura en capas clásica, facilitando pruebas unitarias y mantenimiento.

Capa de presentación ([prog2int.Main](#) y [prog2int.ui](#))

- [AppMenu](#) actúa como Application Controller: crea un único [Scanner](#), arma la cadena de dependencias y mantiene el ciclo de vida del menú principal con manejo de errores básico.
- [MenuHandler](#) encapsula toda la interacción con el usuario, valida entradas mínimas y delega las operaciones reales a los servicios. Para cada opción del menú muestra mensajes contextualizados, recolecta datos y maneja flujos como la creación de pacientes con historia opcional o los listados filtrados.
- El paquete [ui.console](#) ofrece utilitarios de color y contexto

Capa de servicios ([prog2int.Services](#))

- [PacienteServiceImpl](#) aplica validaciones de negocio (formato de DNI, fecha de nacimiento no futura, unicidad por DNI) y coordina operaciones mixtas con historias clínicas mediante el [PacienteHistoriaClinicaService](#) cuando se requiere atomicidad.
- [HistoriaClinicaServiceImpl](#) concentra las reglas para garantizar número de historia con formato [HC-XXXX](#), obligatoriedad de grupo sanguíneo y reutiliza un DAO genérico para simplificar sustitución por mocks en pruebas.
- [PacienteHistoriaClinicaService](#) implementa transacciones compuestas (crear o eliminar paciente + historia) apoyado en [TransactionManager](#) y conexiones JDBC manuales, garantizando *commit/rollback* coherente.
- [GenericService](#) sirve como contrato base para todos los servicios del proyecto, como [PacienteService](#) o [HistoriaClinicaService](#). Esta interfaz utiliza un parámetro genérico `<T>` para que pueda ser reutilizada con cualquier entidad del sistema, manteniendo un diseño más limpio y escalable. En ella se definen los métodos CRUD básicos que deben implementar todos los servicios:
 - `insertar(T entidad)`: para crear un nuevo registro en la base de datos.
 - `actualizar(T entidad)`: para modificar registros existentes.
 - `eliminar(int id)`: permite eliminar registros según su identificador.
 - `getById(int id)`: obtiene una entidad específica por su ID.
 - `getAll()`: devuelve una lista de todas las entidades de ese tipo.

Todos los métodos lanzan una `Exception`, ya que en la capa de servicio se maneja la lógica transaccional, como los `commits` y `rollbacks`, y pueden ocurrir errores de conexión o validación. En resumen, esta interfaz asegura que todos los servicios del sistema implementen las mismas operaciones básicas y facilita la reutilización del código mediante el uso de genéricos.

Capa de acceso a datos ([prog2int.Dao](#))

- [PacienteDAO](#) define todas las sentencias SQL parametrizadas, incluyendo consultas estándar, búsquedas flexibles con [LIKE](#), auditorías que muestran registros eliminados y métodos transaccionales ([insertTx](#), [eliminarTx](#)). El uso sistemático de [LEFT JOIN](#) permite devolver cada paciente junto a su historia (si existe) sin múltiples viajes a la base.
- [HistoriaClinicaDAO](#) (no reproducido íntegramente aquí) replica el enfoque para historias clínicas, con variantes transaccionales y actualizaciones de la clave foránea.
- Ambos DAOs dependen de [DatabaseConnection](#), una fábrica estática que carga el driver de MySQL, valida la configuración y genera conexiones limpias por invocación, habilitando *overrides* por propiedades del sistema para despliegues personalizados.

Capa de dominio ([prog2int.Models](#))

La jerarquía [Base](#) → [Paciente](#)/[HistoriaClinica](#) asegura consistencia de atributos compartidos y documenta las reglas de negocio directamente en las entidades, ofreciendo métodos auxiliares para representaciones textuales y acceso a enumeraciones estandarizadas.

Persistencia, orden de operaciones y transacciones

Estructura física de la base de datos

La base [db_historiaclinica](#) contiene dos tablas: [paciente](#) (datos personales, DNI único y bandera de soft delete) e [historias_clinicas](#) (datos médicos, [nro_historia](#) único, FK obligatoria con restricción [UNIQUE](#) sobre [paciente_id](#)). Este esquema materializa la **relación 1→1** y habilita auditoría mediante los campos [eliminado](#).

Operaciones CRUD y orden recomendado

1. **Alta coordinada:** cuando se crea un paciente con historia, [PacienteServiceImpl](#) invoca al coordinador transaccional para insertar primero la historia (obteniendo su ID), asignarla al paciente y luego persistir al paciente, asegurando que la FK no quede huérfana.
2. **Lecturas enriquecidas:** [PacienteDAO](#) usa [LEFT JOIN](#) para traer historias en una sola consulta, tanto para [getAll](#) como para búsquedas específicas; las variantes de auditoría eliminan el filtro de *soft delete* para revisiones regulatorias.
3. **Actualizaciones:** los servicios validan datos antes de delegar a los DAOs, que actualizan campos básicos sin tocar la bandera [eliminado](#), reservada para operaciones de baja lógica.
4. **Baja lógica:** tanto pacientes como historias se marcan mediante [UPDATE ... SET eliminado = TRUE](#), manteniendo registros para auditoría y posibilitando recuperaciones futuras.

Transacciones, commit y rollback

- `TransactionManager` encapsula el ciclo `startTransaction` → `commit` → `close`, habilitando `rollback` automático si ocurre una excepción o si el recurso se cierra con una transacción activa. Esto evita fugas de conexiones y asegura que `autoCommit` retorne a `true` antes de liberar el recurso.
- `PacienteHistoriaClinicaService` utiliza `TransactionManager` para garantizar **atomicidad** tanto al crear como al eliminar pares paciente-historia: cualquier fallo en la inserción del paciente o la eliminación de la historia dispara `rollback`, preservando consistencia.
- Operaciones independientes (p. ej., actualizar una historia existente) pueden ejecutar su propio `PreparedStatement` fuera de transacción explícita dado que MySQL auto-confirma cada instrucción, lo cual simplifica el flujo para casos simples y mantiene la transacción solo cuando aporta valor.

Validaciones y reglas de negocio

- **Pacientes:** el servicio fuerza la presencia de nombre, apellido y DNI, valida el patrón `\d{7,8}`, impide fechas de nacimiento futuras y consulta al DAO para asegurar unicidad antes de insertar o actualizar (permitiendo que el propio paciente conserve su DNI).
- **Historias clínicas:** el número debe seguir el formato `HC-XXXX` (cuatro dígitos), el grupo sanguíneo no puede ser nulo y se exige una instancia válida antes de cualquier persistencia.
- **Modelos:** el contrato de dominio documenta reglas como la unicidad del DNI, el vínculo 1→1 y la enumeración finita de grupos sanguíneos, lo que guía tanto a desarrolladores como a validadores de datos al momento de extender el sistema.
- **Capa de presentación:** aunque las validaciones principales ocurren en servicios, `MenuHandler` limpia entradas (`trim`), verifica formatos básicos y ofrece mensajes claros cuando las excepciones burbujan, fortaleciendo la experiencia del usuario final.

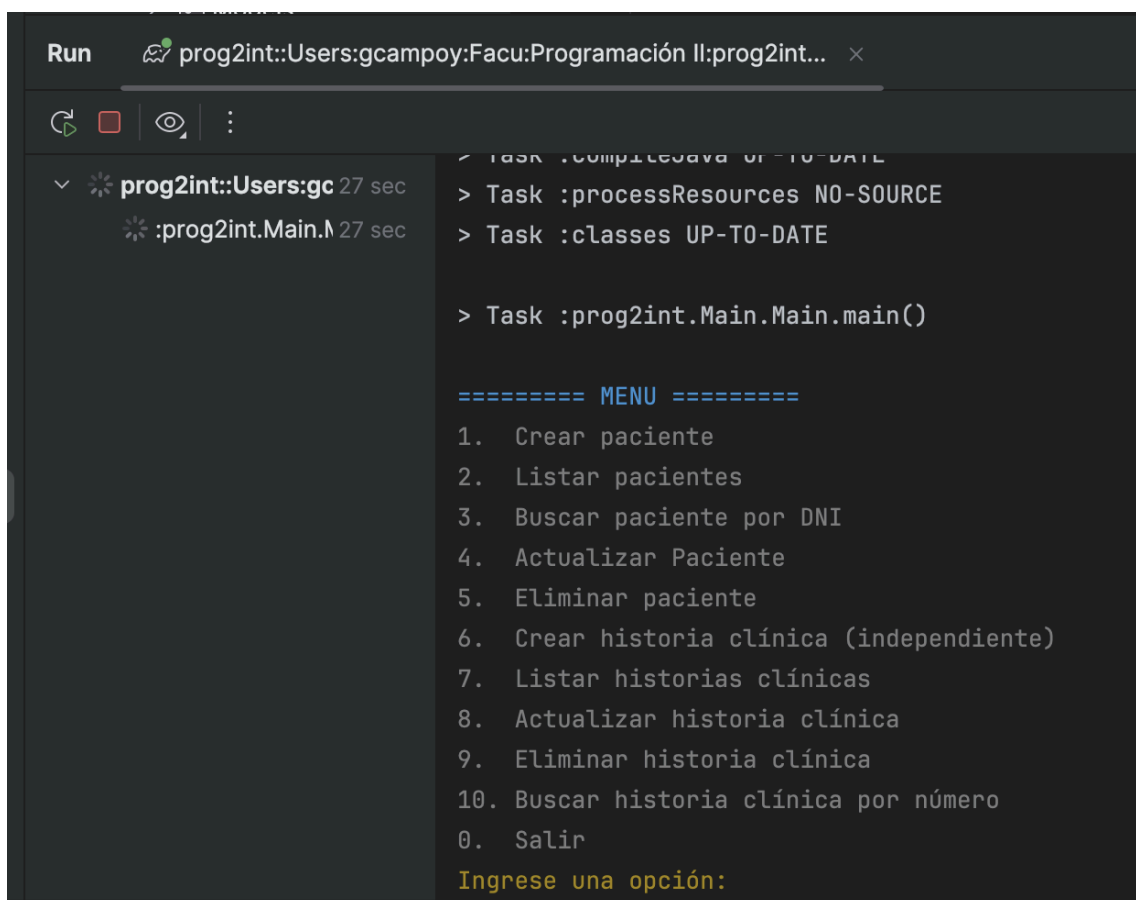
Interacción de usuario y flujo operativo

- El menú principal ofrece **diez opciones** que cubren altas, listados, actualizaciones y bajas, incluyendo operaciones especializadas como eliminar historias de manera segura o buscar pacientes por DNI. La estructura de `switch` en `AppMenu` mantiene el mapa de navegación explícito y fácil de ampliar.
- Cada caso se implementa en `MenuHandler`, donde se describe el flujo de pasos, el manejo de excepciones y los mensajes informativos para el operador. Por ejemplo, la opción “Crear paciente” solicita datos personales y, opcionalmente, invoca el asistente `crearHistoriaClinica()` para asociar la historia dentro del mismo flujo.
- El diseño contempla búsquedas flexibles (por nombre/apellido o por DNI exacto) y listados enriquecidos que muestran los datos médicos cuando están disponibles, alineándose con los requerimientos de auditoría mencionados en la consigna.

Pruebas realizadas

Capturas del menú principal

- Menú principal, con listado de opciones



```
Run  prog2int::Users:gcamboy:Facu:Programación II:prog2int... x
[Icons: Run, Stop, Debug, View, Settings]

▼ * prog2int::Users:gc 27 sec
  * :prog2int.Main.M 27 sec
    > Task :compileJava UP-TO-DATE
    > Task :processResources NO-SOURCE
    > Task :classes UP-TO-DATE

    > Task :prog2int.Main.Main.main()

    ===== MENU =====
    1.  Crear paciente
    2.  Listar pacientes
    3.  Buscar paciente por DNI
    4.  Actualizar Paciente
    5.  Eliminar paciente
    6.  Crear historia clínica (independiente)
    7.  Listar historias clínicas
    8.  Actualizar historia clínica
    9.  Eliminar historia clínica
    10. Buscar historia clínica por número
    0.  Salir

    Ingrese una opción:
```

- Alta de paciente con datos válidos sin historia clínica

```

Ingrese una opción: 1
Nombre: Alberto
Apellido: Einstein
DNI: 34123123
Fecha de nacimiento (YYYY-MM-DD): 1978-12-12
¿Desea agregar historia clínica? (s/n): n
Paciente creado exitosamente con ID: 3

```

- Búsqueda por nombre

```

Ingrese una opción: 2
¿Desea (1) listar todos o (2) buscar por nombre/apellido? Ingrese opcion: 2
Ingrese texto a buscar: Alberto
ID: 3, Nombre: Alberto, Apellido: Einstein, DNI: 34123123

===== MENU =====
1. Crear paciente

```

- Búsqueda por DNI (sin resultados)

```

0. Salir
Ingrese una opción: 3
DNI del paciente: 30485503
Paciente no encontrado.

===== MENU =====

```

- Alta con DNI inválido

```

Nombre: Wylly
Apellido: Cañas
DNI: 0
Fecha de nacimiento (YYYY-MM-DD): 1978-12-12
¿Desea agregar historia clínica? (s/n): n

Error al crear paciente: El DNI debe contener entre 7 y 8 dígitos numéricos sin puntos ni espacios
===== MENU =====

```

Casos de prueba relevantes

1. Alta transaccional

- *Pasos*: Ejecutar opción 1, ingresar datos válidos y confirmar historia.
- *Resultados esperados*: mensaje de éxito y verificación en base.

2. Búsqueda por DNI inexistente

- *Pasos*: Opción 3 del menú principal (buscar) con un DNI no cargado.
- *Resultados*: mensaje “Paciente no encontrado” sin excepciones.

3. Baja lógica coordinada

- *Pasos*: Opción segura para eliminar historia (según diseño futuro) y luego verificar listados.
- *Resultados*: registros excluidos de listados activos, visibles en auditorías.

Consultas SQL útiles

- `SELECT * FROM paciente WHERE eliminado = FALSE;`
- `SELECT * FROM historias_clinicas WHERE paciente_id IS NOT NULL;`
- `SELECT p.*, hc.* FROM paciente p LEFT JOIN historias_clinicas hc ON p.id = hc.paciente_id;`

Conclusiones y mejoras futuras

El proyecto evidencia una separación de responsabilidades madura y cumple los objetivos académicos planteados: **arquitectura en capas**, uso de **patrones (DAO, Service, Factory)** y gestión **transaccional explícita**. Para iteraciones futuras se podría evaluar profundizar en pruebas automatizadas, exponer la lógica mediante una API REST y reforzar las operaciones de eliminación segura de historias para cubrir todos los escenarios previstos por las reglas de negocio.

Fuentes y herramientas utilizadas

- *Proyecto base subido por personal docente a la plataforma*
- *Uso de IA*
 - a. **gemini**: validaciones sobre estructura de código, enfoque de separación en capas
 - b. **copilot**: asistente con plugin en IDE, se utilizó entre otras cosas para refactorizar código, evidenciar buenas prácticas
 - c. **codex**: full scan del proyecto para evidenciar oportunidades futuras no implementadas, mejorar documentación en general, validar que el contexto dado en readme, historias_de_usuario, es legible y útil incluso para una IA, lo cual refuerza la necesidad de buena documentación y entendimiento de requerimientos por parte del equipo
 - d. **chatGPT**: consultas esporádicas, sobre problemas puntuales